Task 5: Storage Classes: The Same Counter Four Ways (7 Marks)

Task 5.1: Four Implementations (3 Marks)

(a) Automatic local variable

```
#include <stdio.h>
int token_auto(void) { int token = 1000; return ++token; }
int main(void) {
    printf("(a) auto  : ");
    for (int i = 0; i < 3; i++) printf("%d ", token_auto());
    printf("\n");
    return 0;
}
```

Output

```
(a) auto  : 1001 1001 1001
```



The variable is created and re-initialised to 1000 on every call, so every call returns 1001. An automatic variable cannot remember anything between calls.

(b) Static local variable

```
#include <stdio.h>
int token_static(void) { static int token = 1000; return ++token; }
int main(void) {
    printf("(b) static: ");
    for (int i = 0; i < 3; i++) printf("%d ", token_static());
    printf("\n");
    return 0;
}
```

Output

```
(b) static: 1001 1002 1003
```



The static variable is initialised once (before the program starts) and keeps its value for the whole run, while still being visible only inside the function.

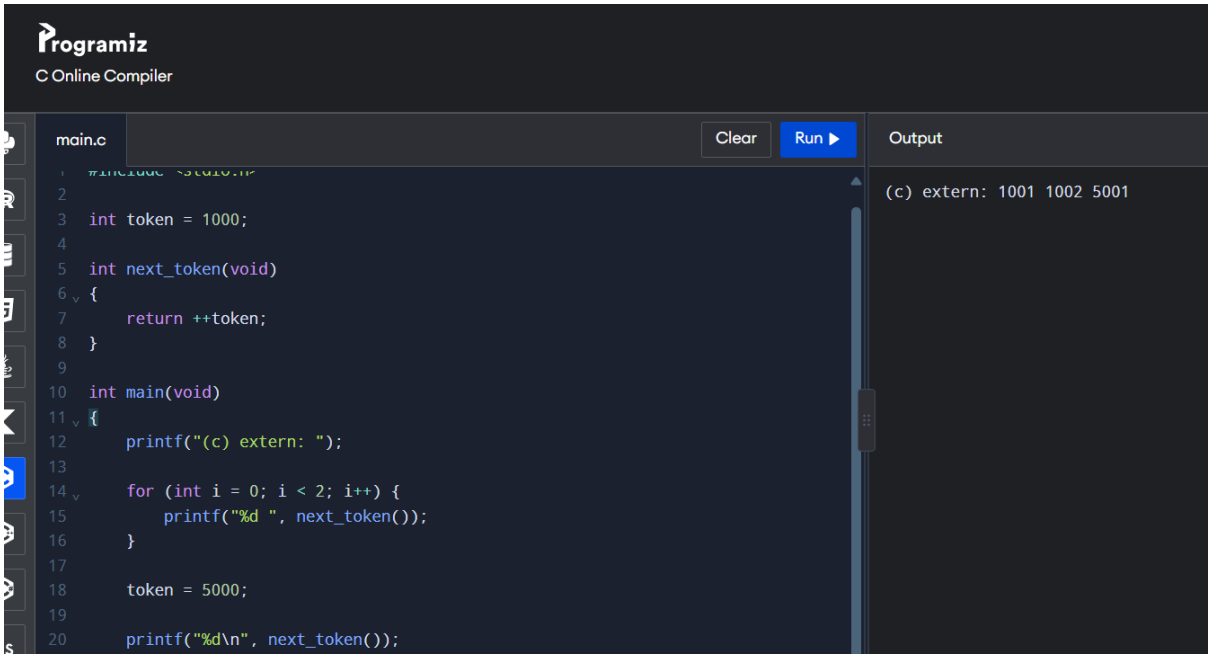
(c) Global variable shared across two files with extern

```
/* file 1: defines the shared global */
int token = 1000;
int next_token(void) { return ++token; }
```

```
/* file 2: uses the same global through extern */
#include <stdio.h>
extern int token;
extern int next_token(void);
int main(void) {
    printf("(c) extern: ");
    for (int i = 0; i < 2; i++) printf("%d ", next_token());
    token = 5000;               /* any file can corrupt it! */
    printf("%d\n", next_token());
    return 0;
}
```

Output

```
(c) extern: 1001 1002 5001
```



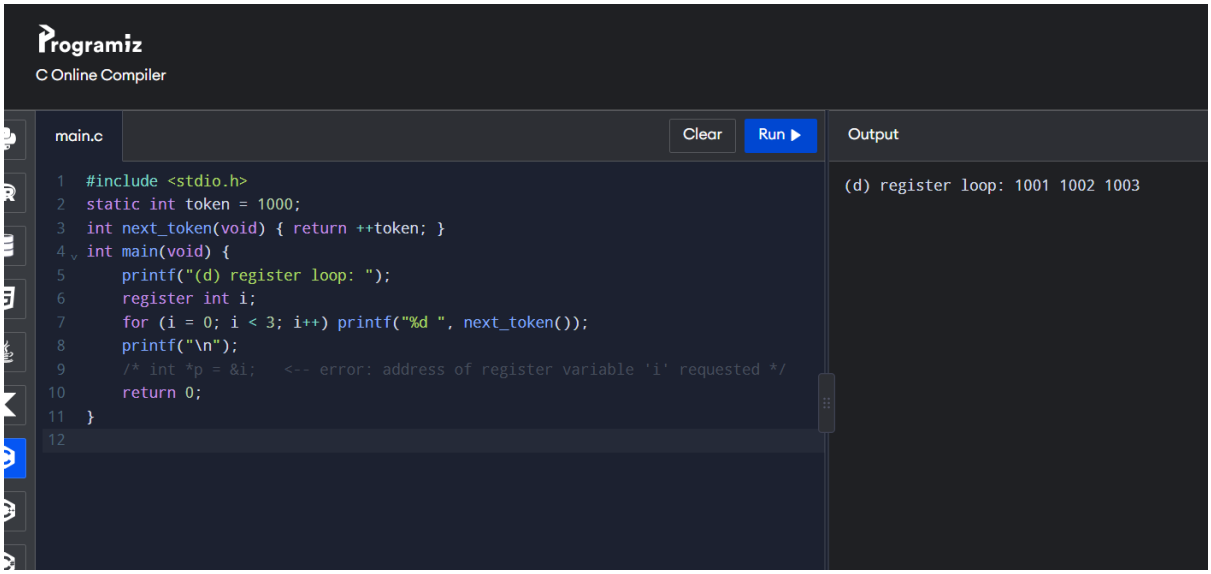
File 1 defines token; file 2 refers to the same variable through extern. The counter works across files, but file 2 was also able to set token = 5000, so the next token became 5001. This shows the accidental-modification risk. (Build: gcc c_counter.c c_main.c.)

(d) Register loop variable in the printing loop

```
#include <stdio.h>
static int token = 1000;
int next_token(void) { return ++token; }
int main(void) {
    printf("(d) register loop: ");
    register int i;
    for (i = 0; i < 3; i++) printf("%d ", next_token());
    printf("\n");
    /* int *p = &i;   <-- error: address of register variable 'i' requested */
    return 0;
}
```

Output

```
(d) register loop: 1001 1002 1003
```



The loop counter i is a register variable (a request to keep it in a CPU register for speed). It controls the loop only; the token numbers still come from a counter that lives elsewhere (here, a file-scope static). A register variable cannot itself carry the counter between calls.

Version	Tokens recorded	Correct?
(a) automatic	1001 1001 1001	No (state is lost)
(b) static local	1001 1002 1003	Yes
(c) global with extern	1001 1002, then 5001 after outside tampering	Yes, but not protected
(d) register loop variable	1001 1002 1003	Yes (counter is separate from the register variable)

Task 5.2: Change the Requirement (2 Marks)

Two independent counters (veg and non-veg), each starting at 1001.

Static version

```
#include <stdio.h>
int next_veg(void)    { static int t = 1000; return ++t; }
int next_nonveg(void) { static int t = 1000; return ++t; }
int main(void) {
    int v1 = next_veg(), v2 = next_veg();
```

```
    int n1 = next_nonveg(), n2 = next_nonveg();
    int v3 = next_veg();
    printf("veg: %d %d  nonveg: %d %d  veg: %d\n", v1, v2, n1, n2, v3);
    return 0;
}
```

Output

```
veg: 1001 1002  nonveg: 1001 1002  veg: 1003
```

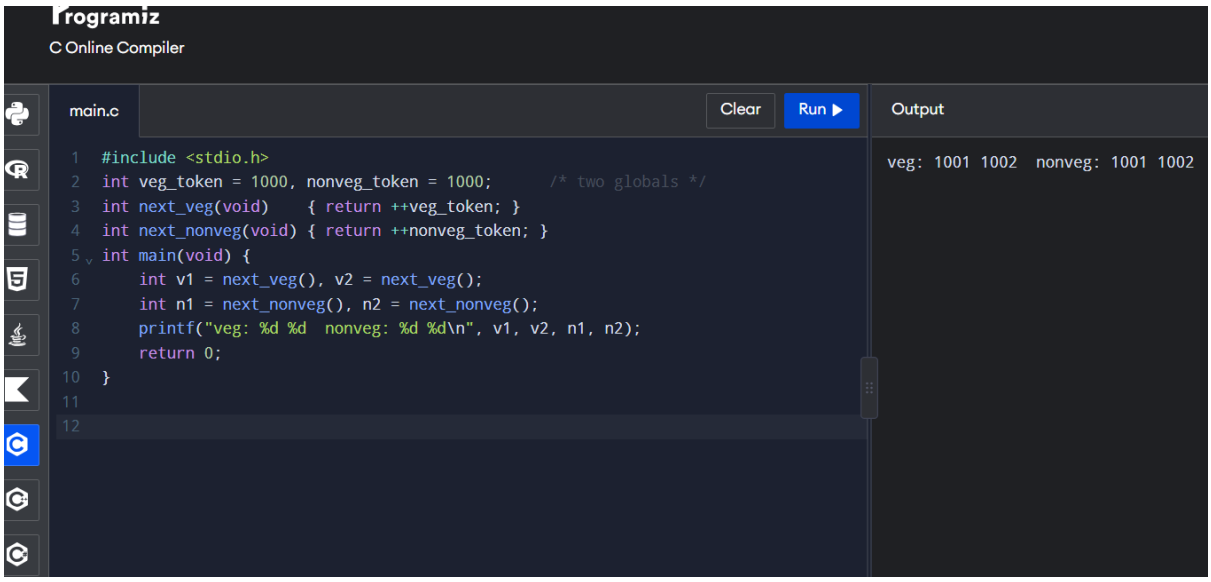


Global version

```
#include <stdio.h>
int veg_token = 1000, nonveg_token = 1000;      /* two globals */
int next_veg(void)    { return ++veg_token; }
int next_nonveg(void) { return ++nonveg_token; }
int main(void) {
    int v1 = next_veg(), v2 = next_veg();
    int n1 = next_nonveg(), n2 = next_nonveg();
    printf("veg: %d %d  nonveg: %d %d\n", v1, v2, n1, n2);
    return 0;
}
```

Output

```
veg: 1001 1002  nonveg: 1001 1002
```



Version	What had to change	Approximate code change
(a) automatic	Cannot meet the requirement at all: neither counter can remember anything. The storage class itself must change (to static, global or pointer-based) first.	Full redesign
(b) static local	A second function with its own static variable (next_veg, next_nonveg); all call sites change to choose the right function.	+1 function (3 lines); call sites edited
(c) global / extern	A second global, a second function, and an extern declaration for each new name in every file that uses it.	+1 global, +1 function, +1 extern per file (about 5 lines or more)
(d) register loop variable	The loop variable is unaffected; the change happens in whichever counter version it prints from.	0 lines in the loop itself

In (b) and (c) the **function is duplicated** for every new counter. With 10 counters there would be 10 near-identical functions, which does not scale.

Task 5.3: Compare the Storage Classes (2 Marks)

	auto	static local	global (extern)	register
Scope	Block (function)	Block (function)	File scope; visible in other files through extern	Block
Lifetime	While the block runs	Whole program	Whole program	While the block runs
Initial value	Indeterminate (garbage) unless initialised	0 by default; initialised once, before main	0 by default; initialised once	Indeterminate unless initialised
Accidental-modification risk	Low (private) but loses state	Low (private to the function), but hidden state and not thread-safe	High: any function in any file can change it; name clashes	Low (private)

Why the address of a register variable cannot be taken. register asks the compiler to keep the variable in a CPU register. Registers do not have memory addresses, so the address-of operator & has nothing to return. The C standard therefore makes & on a register variable a constraint violation. GCC reports it:

Output

```
bad.c:1:1: error: address of register variable 'i' requested
```

Modern compilers ignore register as a speed hint, because they allocate registers better themselves, but the language rule still applies.

Complete the Code (TODO 1)

```
#include <stdio.h>

/* Issues the next mess token number, starting from 1001. */
int next_token(void)
{
    static int token = 1000;     /* TODO 1: initialised once, keeps its value */
    return ++token;
}

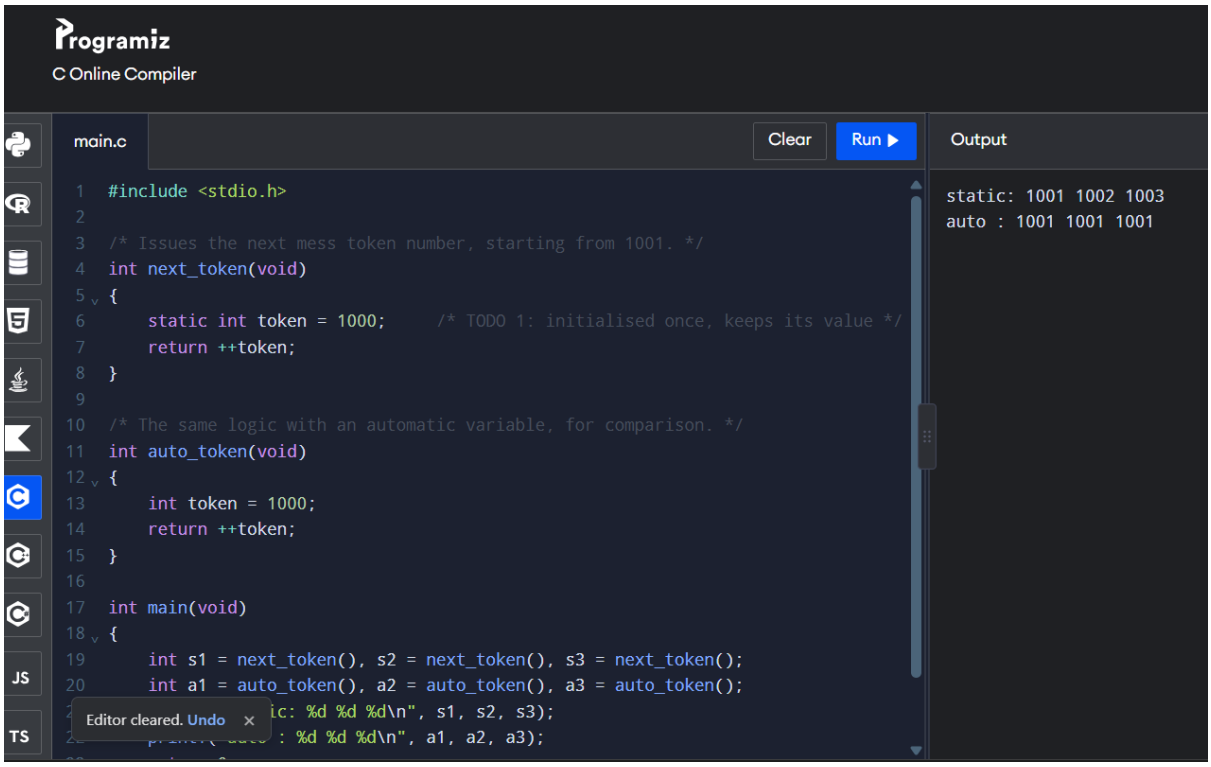
/* The same logic with an automatic variable, for comparison. */
int auto_token(void)
{
    int token = 1000;
```

```
    return ++token;
}

int main(void)
{
    int s1 = next_token(), s2 = next_token(), s3 = next_token();
    int a1 = auto_token(), a2 = auto_token(), a3 = auto_token();
    printf("static: %d %d %d\n", s1, s2, s3);
    printf("auto : %d %d %d\n", a1, a2, a3);
    return 0;
}
```

Output

```
static: 1001 1002 1003
auto : 1001 1001 1001
```

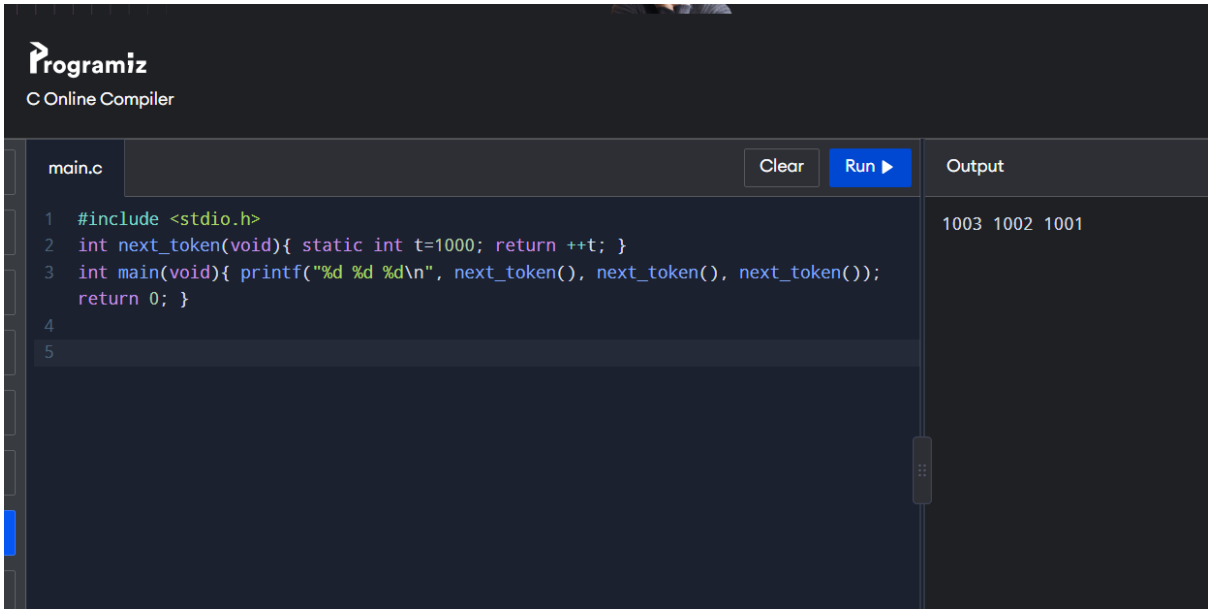


Order-of-evaluation experiment

```
#include <stdio.h>
int next_token(void){ static int t=1000; return ++t; }
int main(void){ printf("%d %d %d\n", next_token(), next_token(), next_token()); return 0; }
```

Output

```
1003 1002 1001
```



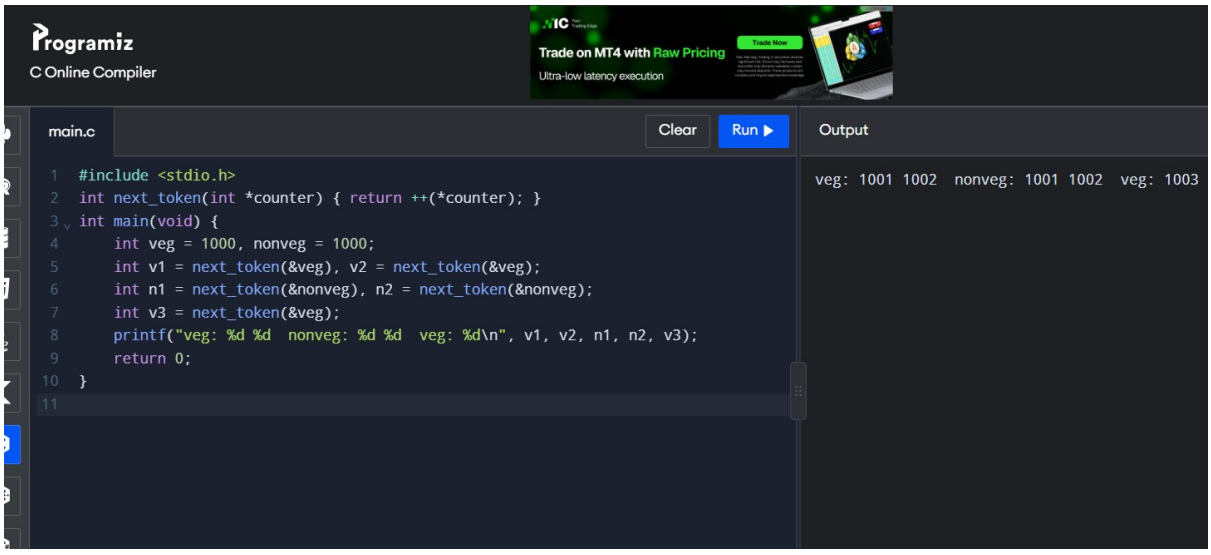
The three calls in one printf are the function arguments. The C standard does **not specify** the order in which function arguments are evaluated, so the compiler may call next_token() in any order. GCC on x86-64 usually evaluates the arguments from right to left, so the rightmost call gets 1001 and the leftmost gets 1003. Another compiler (or another optimisation level) may print 1001 1002 1003. The program is not undefined, but its output is unspecified and must not be relied on. The fix is to make separate statements (int a = next_token(); int b = next_token(); ...) as in the main program.

Alternative Concept and Complexity Analysis: Counter Owned by the Caller

```
#include <stdio.h>
int next_token(int *counter) { return ++(*counter); }
int main(void) {
    int veg = 1000, nonveg = 1000;
    int v1 = next_token(&veg), v2 = next_token(&veg);
    int n1 = next_token(&nonveg), n2 = next_token(&nonveg);
    int v3 = next_token(&veg);
    printf("veg: %d %d  nonveg: %d %d  veg: %d\n", v1, v2, n1, n2, v3);
    return 0;
}
```

Output

```
veg: 1001 1002  nonveg: 1001 1002  veg: 1003
```



Aspect	Result
Time	O(1) per call in all versions: one increment (the pointer version adds one dereference).
Space	One `int` (4 bytes) per counter, plus an 8-byte pointer argument on the stack during the call. No hidden storage in the function.
Lifetime	Decided by the caller: the counter lives as long as the caller's variable (a local in `main` here), not for the whole program. It can be reset, saved or destroyed explicitly.
Two independent counters	Trivial: declare another `int` and pass its address. One line of change and the function body stays untouched, whereas the static and global versions need a new function each.
Safety	Only code that holds the pointer can change the counter, so there is no global exposure; the function is re-entrant and thread-friendly. The caller must pass a valid pointer.

Evaluation. The pointer version is the best of all: it scales to any number of counters without duplication, and it avoids the hidden state of static and the open access of global variables.